

W4D4 – Progetto Finale Cybersecurity Analyst

Studente: Emanuele cestari

Corso: Epicode – Cybersecurity Analyst

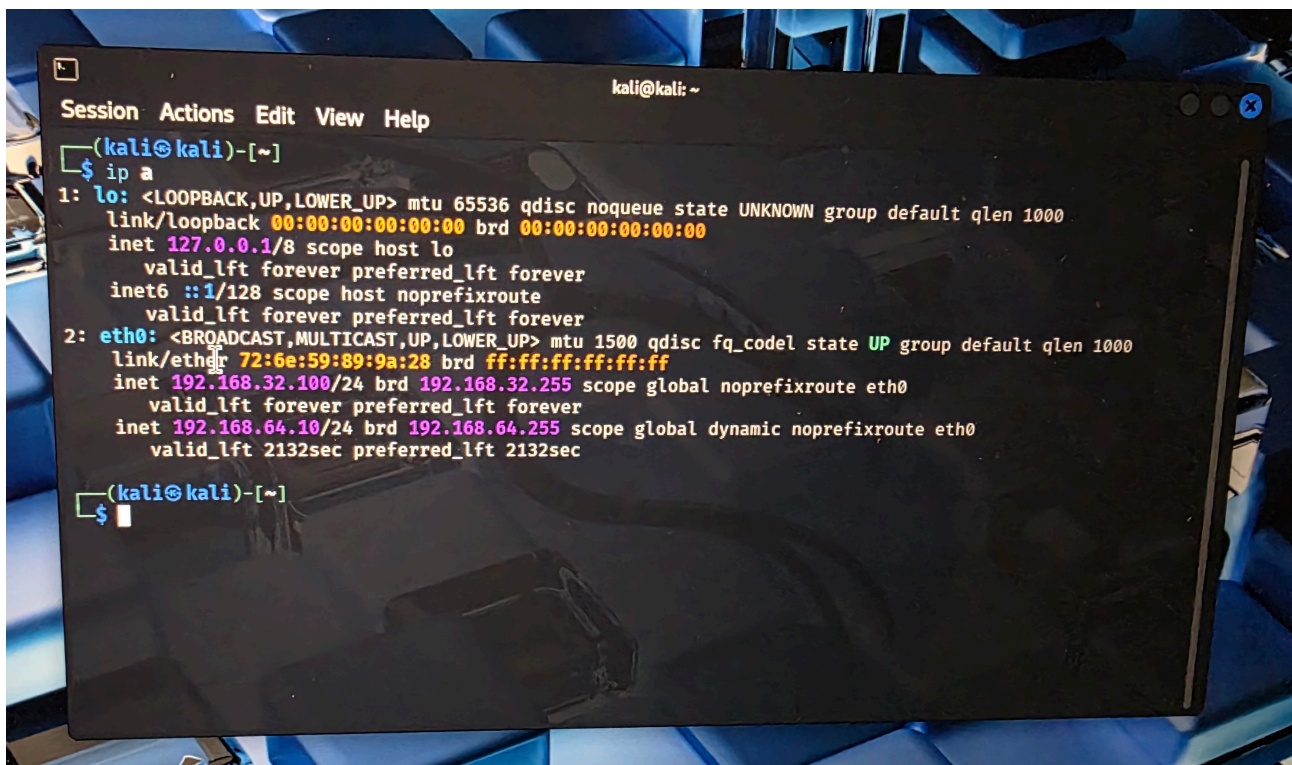
Obiettivo dell'esercizio

L'obiettivo di questo esercizio è simulare un'architettura client-server in un ambiente virtuale. In questo scenario, un client Windows richiede una risorsa web tramite l'hostname "**epicode.internal**", che viene risolto tramite DNS verso un server Kali Linux. Il traffico viene intercettato con Wireshark per confrontare le differenze tra HTTP e HTTPS.

Configurazione dell'ambiente

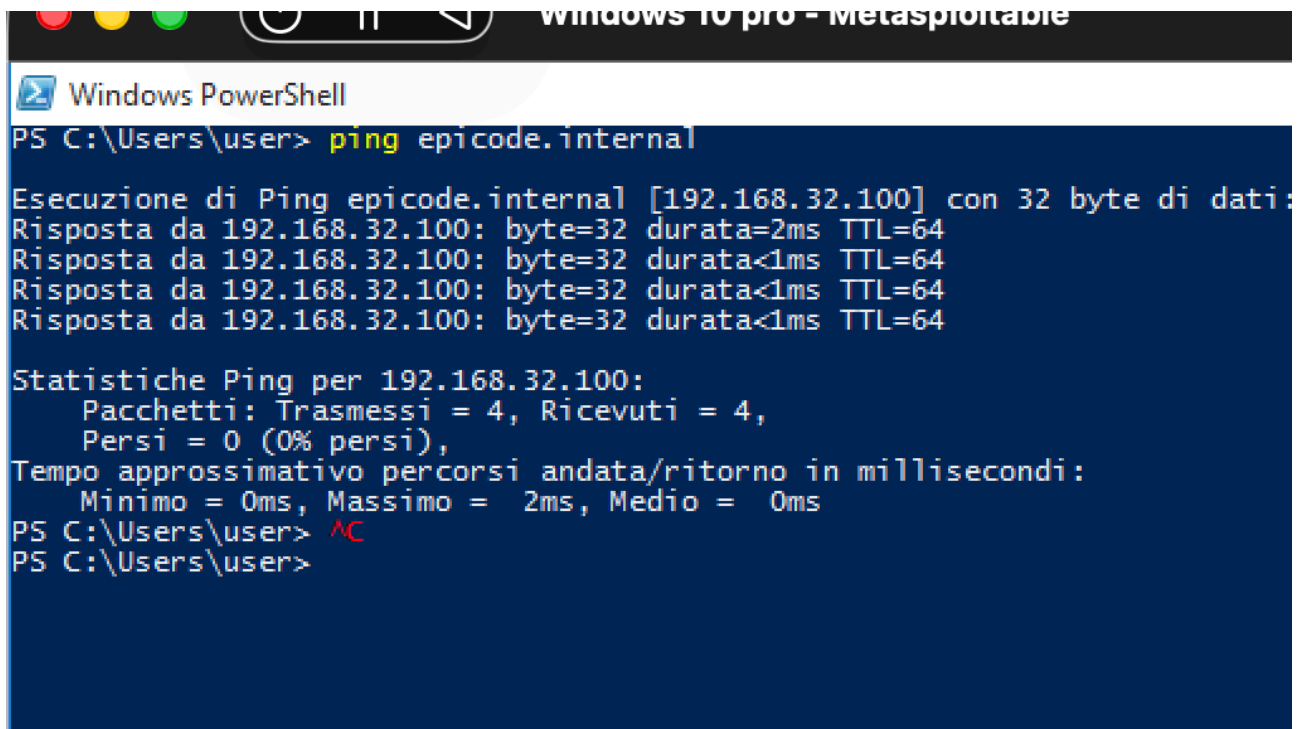
- **Kali Linux (Server):** 192.168.32.100
- **Windows 10 (Client):** 192.168.32.101
- **Rete:** /24
- **Servizi:** DNS (hostname "**epicode.internal**"), HTTP, HTTPS

Kali linux IP



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 72:6e:59:89:9a:28 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.32.100/24 brd 192.168.32.255 scope global noprefixroute eth0  
        valid_lft forever preferred_lft forever  
    inet 192.168.64.10/24 brd 192.168.64.255 scope global dynamic noprefixroute eth0  
        valid_lft 2132sec preferred_lft 2132sec  
(kali@kali)-[~]  
$
```

Il comando *ping epicode.internal*, eseguito dal client Windows, dimostra la corretta risoluzione del nome verso l'indirizzo IP del server Kali. Il server Kali Linux funge anche da DNS server, risolvendo il nome *epicode.internal* nell'indirizzo IP 192.168.32.100.



```
Windows PowerShell
PS C:\Users\user> ping epicode.internal

Esecuzione di Ping epicode.internal [192.168.32.100] con 32 byte di dati:
Risposta da 192.168.32.100: byte=32 durata=2ms TTL=64
Risposta da 192.168.32.100: byte=32 durata<1ms TTL=64
Risposta da 192.168.32.100: byte=32 durata<1ms TTL=64
Risposta da 192.168.32.100: byte=32 durata<1ms TTL=64

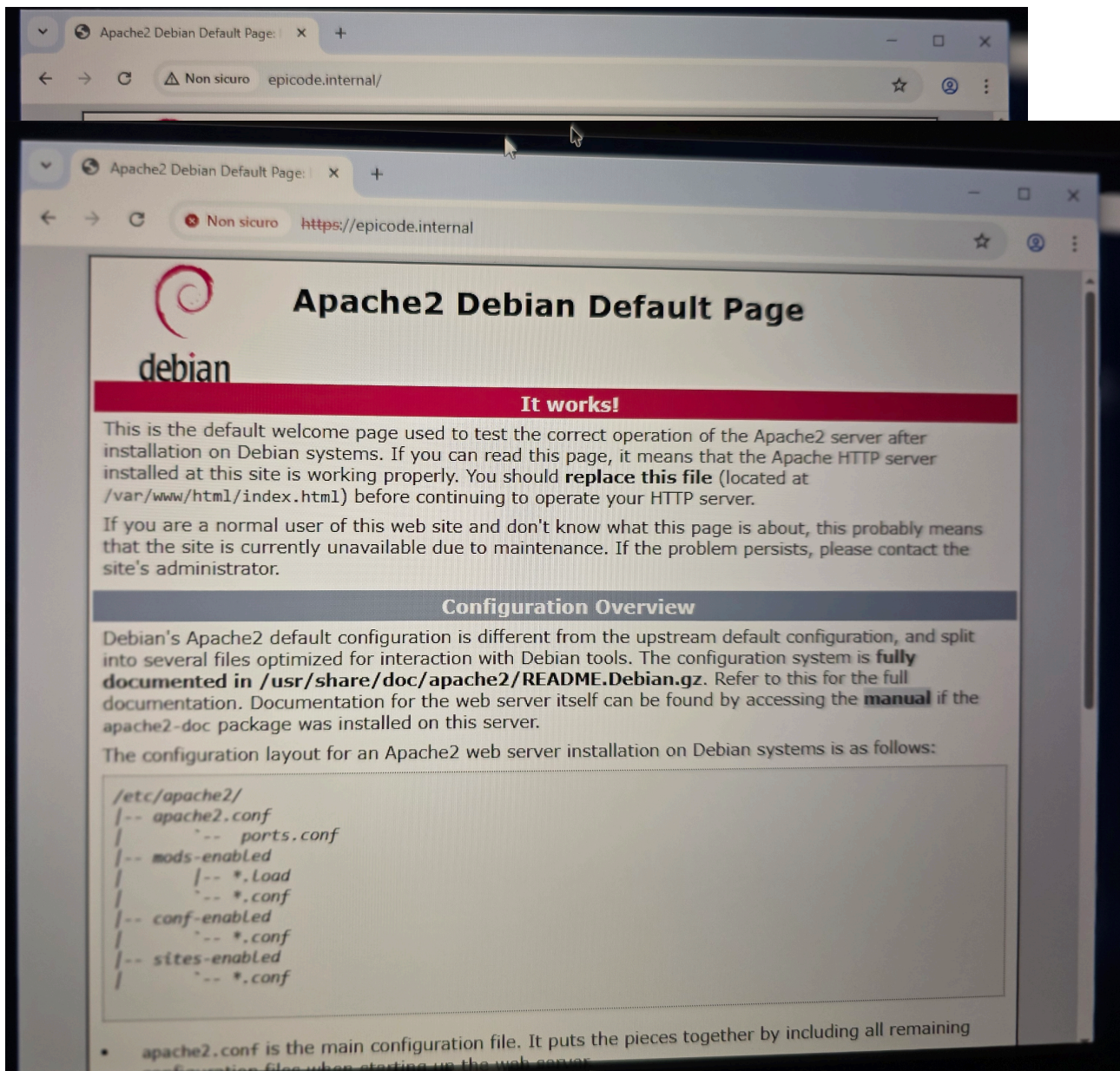
Statistiche Ping per 192.168.32.100:
    Pacchetti: Trasmessi = 4, Ricevuti = 4,
    Persi = 0 (0% persi),
Tempo approssimativo percorsi andata/ritorno in millisecondi:
    Minimo = 0ms, Massimo = 2ms, Medio = 0ms
PS C:\Users\user> ^C
PS C:\Users\user>
```

Il browser del client Windows si connette correttamente alla risorsa web tramite HTTP, ottenendo la pagina di default del server Apache2.

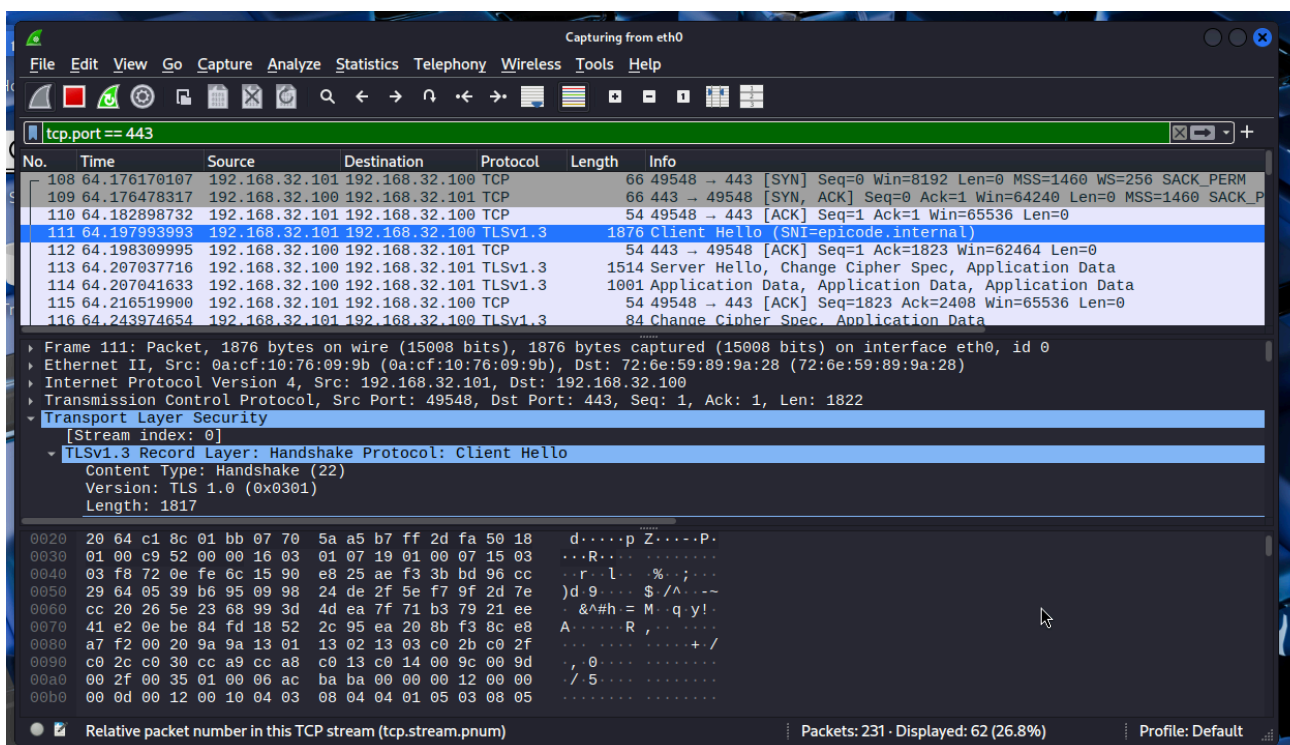
Il traffico HTTP è stato catturato con Wireshark sull'interfaccia *eth0*, applicando il filtro *http*. Wireshark mostra chiaramente la richiesta HTTP in chiaro, con il metodo GET, l'header *Host* e il contenuto della risposta.

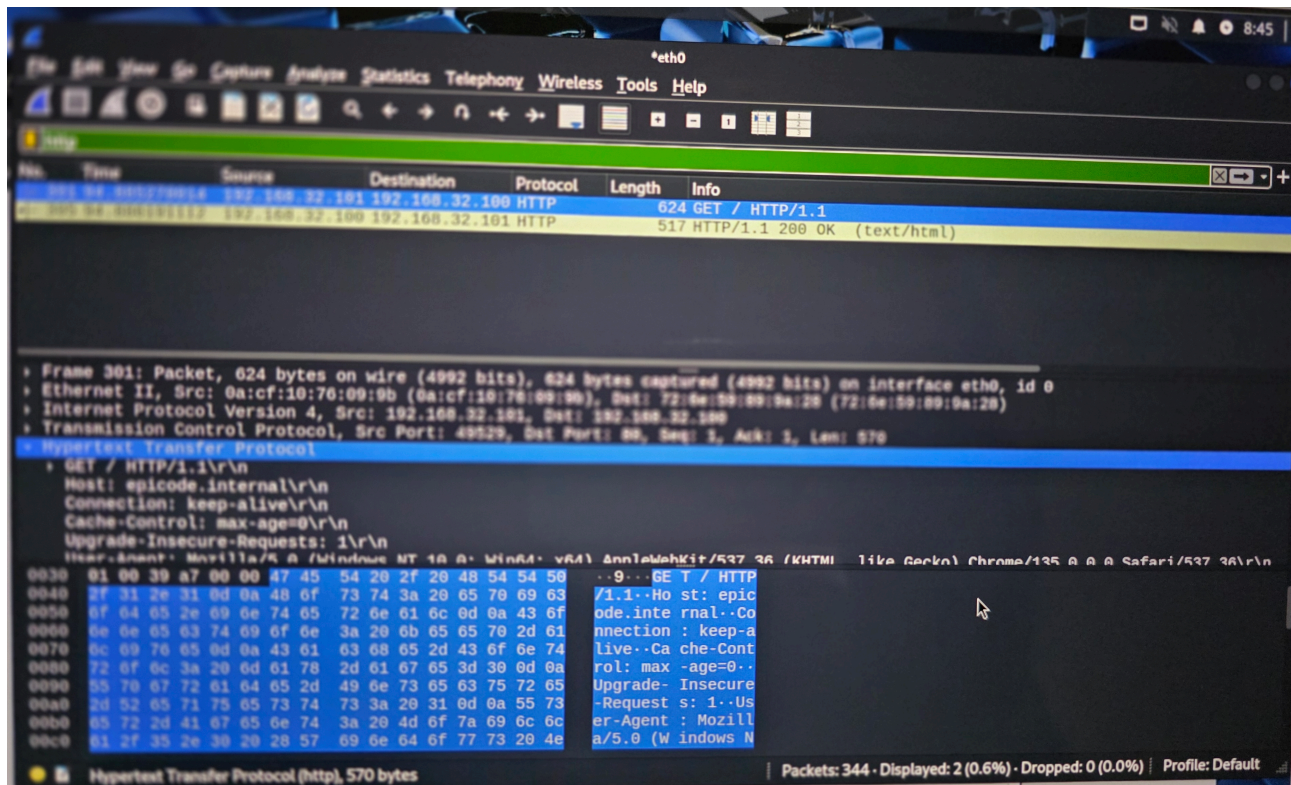
Accesso al servizio HTTPS

Il client accede al servizio HTTPS tramite certificato autofirmato. La comunicazione avviene sulla porta 443.



Analisi del Traffico HTTPS con Wireshark





Per analizzare il traffico HTTPS, è stato utilizzato il filtro "tcp.port == 443". Durante l'analisi, Wireshark mostra l'handshake TLS, che inizia con il Client Hello contenente l'SNI "epicode.internal". Questo handshake è fondamentale perché il client e il server negoziano i parametri di sicurezza e stabiliscono una chiave di sessione, che verrà utilizzata per cifrare la comunicazione. Di conseguenza, il contenuto applicativo risulta cifrato e non leggibile.

Conclusione:

È stata simulata un'architettura client-server in un ambiente virtuale. Il client Windows (192.168.32.101) ha richiesto una risorsa al server Kali Linux (192.168.32.100) utilizzando l'hostname "epicode.internal". La risoluzione del nome è avvenuta correttamente tramite DNS. Il traffico di rete è stato intercettato con Wireshark, che ha evidenziato gli indirizzi MAC di sorgente e destinazione.

Nel caso del traffico HTTPS, il contenuto della comunicazione applicativa è stato cifrato tramite TLS, rendendolo illeggibile. Al contrario, nel caso del traffico HTTP, la richiesta è risultata visibile in chiaro, mostrando dettagli come il metodo (GET), l'intestazione Host e il percorso.

Questo dimostra come HTTPS garantisca confidenzialità dei dati, mentre HTTP esponga il contenuto del traffico.

HTTPS protegge la comunicazione da attacchi di sniffing e man-in-the-middle, impedendo la lettura del payload applicativo.

Confronto HTTP vs HTTPS

HTTP: traffico in chiaro, vulnerabile a sniffing e man-in-the-middle.

HTTPS: traffico cifrato tramite TLS, garantisce confidenzialità e integrità dei dati.
L'analisi dimostra l'importanza dell'uso di HTTPS per la sicurezza delle comunicazioni.